Repeat Viewing Beats Collaborative Signal on sweet.tv: A Leakage-Free LightGBM Hybrid over Personal Frequency, Personalized PageRank and Association Rules for Top-5 TV Programmes

Student: Sergey Emelyanov Team: Individual Email: sbemelyanov@edu.hse.ru Date: 2026-10-24
Assignment: Final Exam Quiz (AVITO) — TV Program Recommendation on sweet.tv Kaggle: pochtasergeia
Repository: https://github.com/Fighter90/sweettv-recommender

Abstract

The task is to predict, for 1,260 sweet.tv users, the five programmes they will watch most often in weeks 31–42, scored by MAP@5 against a top-5 built from airings watched for at least 80 % of their length. I joined 5.82 M viewing intervals to the 10.8 % of schedule rows that carry a programme id, reproduced the organisers' answer rule on a weeks 11–20 → 21–30 hold-out, and compared a popularity baseline with personal frequency, Personalized PageRank (PPR), association rules, ALS matrix factorisation, item-kNN and content-based filtering before re-ranking their candidates with LightGBM. On the 1,150 test users with a full hold-out top-5, the best single method (PPR, $\alpha = 0.5$) reaches MAP@5 0.394 and popularity 0.305, while the final hybrid ensemble reaches 0.447; on Kaggle it returned private 0.44801 (public 0.46135). On this platform, most of the signal is repeat viewing of the same news, kids' and entertainment shows, so a hybrid that keeps the user's own history as the backbone outperforms pure collaborative filtering.

Index Terms — recommender systems, implicit feedback, personalized PageRank, learning to rank, TV viewing logs.

1. Introduction

Problem statement. The competition [1] gives a view log (`user_id, vsetv_id, start_time, stop_time, duraton`) for weeks 11–30 of 2020 and broadcast schedules for weeks 11–42. A view is a channel interval, not a programme, so interactions must be built by joining views to airings. The answer for weeks 31–42 is defined on the Data page: drop `tv_show_id = 0`, compute per user and airing *screen time = watched time / programme duration*, keep airings with screen time ≥ 0.8, count them per programme and take the five most frequent, in order.

Motivation. My first local truth (any overlap ≥ 300 s) gave the popularity baseline MAP@5 0.727. That is far from the 0.4–0.46 range of real submissions, so the metric was not measuring the competition's target. With the official rule the same baseline scores 0.305, and the hold-out estimate of the first hybrid (0.440) landed near its Kaggle private score (0.448). Before tuning any model, the evaluation itself has to be validated.

Concrete example. User 8377619604347126107 (90th percentile of activity) watched *ТСН. Неделя* 59 times and *Факты. День* 22 times at ≥ 80 % screen time in weeks 11–20. In weeks 21–30 the top-5 was ТСН. Неделя, Сегодня. Итоги, Факты. День, Завтрак с 1+1 and Свинка Пеппа. The hybrid returns exactly these five (AP@5 = 1.000), popularity hits four of them (0.710) and ALS three (0.550), Table 3.

Contributions. (i) A leakage-free pipeline that reproduces the organisers' answer rule locally, with 124 logged attempts in which each method is first evaluated individually, then combined. (ii) A hybrid (candidate generation + LightGBM) that improves MAP@5 by +0.053 over the best single method on the same users, and a Kaggle private score of 0.44801 (public 0.46135). (iii) An independent reproduction on a second machine, which shows which of these numbers are stable and which depend on the numeric library (§4, Table 4).

2. Related Work

[2] Hu, Koren and Volinsky: confidence-weighted ALS for implicit feedback, my MF method (`implicit` library). [3] Haveliwala: topic-sensitive (personalised) PageRank, a random walk with restart to a preference set, my PPR on the user–programme graph. [4] Agrawal and Srikant: support/confidence association rules, used here as pairwise rules A → B over users' programme sets. [5] Sarwar et al.: item-based CF with cosine similarity. [6] Lops et al.: content-based recommenders built from item attributes, my TF-IDF CBF. [7] Ke et al.: LightGBM, the gradient-boosted re-ranker. [8] Burges: LambdaRank/LambdaMART, the listwise objective I compared with a pointwise binary one. [9] Hamner: the reference `apk/mapk` code used by Kaggle MAP@K metrics.

Alternatives considered. (a) Pure MF as the final model: rejected because it scores 0.235 against 0.392 for simply repeating the user's own most-watched shows (attempts 66 vs 61). (b) A fraction-of-duration watch threshold other than 0.8 for the *answer*: not an option, since the organisers fix it; for *features* 0.5 and 0.95 were tried (0.5 is best as a single method, 0.392 vs 0.383). (c) Lift-ranked association rules: they collapse to 0.012 because lift rewards rare programmes (attempt 84).

3. Method

Approach. I build interactions with an interval join, then evaluate everything on one temporal hold-out with the official answer rule. Single methods produce scores over the candidate pool and the top-5 is read off directly. The hybrid takes the union of their candidates and learns a ranking from history-only features.

Pipeline.

1. **Join** (`prep.py`). Keep schedule rows with `tv_show_id ≠ 0`; per channel, binary-search each view against airing starts/ends; overlap = min(ends) − max(starts) > 0; sum the overlap of several view fragments of the same airing. The week of an airing is taken from its start time, not from the schedule file it is stored in (the weeks 11–20 file also holds airings of 2020-05-18, week 21). Result: 3,014,129 user–airing rows for 4,895 users and 4,363 programmes, 20.4 % of them with screen time ≥ 0.8.
2. **Candidate pool.** Programmes with an id that air in the target window according to its schedule (6,376 for weeks 21–30; 6,642 for 31–42). Repeats of already-watched programmes are allowed for every method, because the answer counts repeat viewing. Every list is padded with the same popularity list. This is also the cold-start fallback: 2 of the 1,260 test users have no labelled history and receive it unchanged.

3. **Single methods** (`methods.py`). Popularity: number of ≥ 0.8 airings per programme in the history. Personal frequency: the user's own count of ≥ τ airings. ALS [2] on log(1+count), confidence 1 + α·value. Item-kNN [5] with cosine and shrinkage. PPR [3]: restart probability α at the user's own programmes, item → user → item transitions on log-count edges, 20 power iterations. Association rules [4]: confidence(A → B) over programme sets, minimum support, score = max over the user's antecedents. CBF [6]: TF-IDF over category, genres, directors, actors and main channel; profile = log-count-weighted sum. Each method outputs a score; its top-5 is the answer.
4. **Hybrid** (`features.py`, `ranker.py`). Candidates = the user's own history in the pool, the top-30 of PPR, ALS, AR and CBF, and the top-20 popular programmes (189.5 per user; they contain 85.2 % of the true top-5 items). About 100 features: own counts at several thresholds and recencies, weekly lags, catch rate = hits / airings, number and timing of airings in the target window (schedule only), channel and hour-of-day affinity, show stickiness, generator scores and ranks, full-pair PPR at α 0.5/0.7. LightGBM [7] learns P(item ∈ true top-5). Per-user ranks of several models are averaged.
5. **Final model** (attempt 123). A rank average of three seed bags: binary, 31 leaves, 400 rounds, learning rate 0.05, 5 seeds; binary, learning rate 0.03, 700 rounds, 3 seeds; L2 regression on log(1+future count), 3 seeds. Trained on the 11–20 → 21–30 hold-out of all 4,030 users plus two extra temporal splits (11–24 → 25–30, 11–17 → 18–24). It scores weeks 31–42 from features of weeks 11–30.

Tools & libraries. Python 3.13.16, numpy 2.5.3, pandas 3.0.5, scipy 1.18.1, scikit-learn 1.9.1 (TF-IDF), implicit 0.7.3 (ALS) [10], LightGBM 4.7.0 [11], matplotlib 3.11.2. AI agents (§6): Claude Code for development, OpenCode v2.1.2 [12] with GLM-5.3-Flash for the reproduction on my Mac.

Key design decisions. (1) *Validate the metric first*: the truth rule is copied from the Data page, and ties in the local truth are broken by id because the organisers' tie rule is not published. (2) *User-grouped 3-fold CV* for the hybrid: the scored user's rows never appear in the training folds, and the extra temporal splits drop the held-out users too. (3) *Longer history at test time*: a model trained on 5-week features scored 10-week features better than matched 5-week ones (all users 0.4024 vs 0.3972, attempts 26 vs 25). So the final model, trained with 10–14-week histories, is applied to the full 20 weeks.

Configuration. Hold-out: history 11–20, truth 21–30. LightGBM: feature_fraction 0.8, bagging 0.8, λ2 = 1, min_data_in_leaf 50. ALS f = 16, reg 0.05, α = 10, seed 42. PPR α = 0.5, 20 iterations. AR min support 50. Fold seed 0. Hardware: 4 vCPU Intel Xeon 2.1 GHz, 15 GB RAM, numpy with OpenBLAS (all numbers in Tables 1–3).

4. Experiments

Setup. Table 1 gives the data accounting (from `results/data_stats.txt`). MAP@5 uses the slide/Kaggle formula AP@5 = Σ_k P@k·rel(k) / min(|R|, 5). Two user sets: the 1,150 *test users* that have a full top-5 in weeks 21–30, and all 4,030 such users (lower variance; the standard error is about 0.005).

Quantity	Value
View log	5,821,987 intervals; 4,914 users; 202 channels; median 396 s, max 342,145 s
Schedules (rows / with id)	11–20: 1,074,235 / 115,613; 21–30: 1,074,394 / 121,505; 31–42: 1,295,378 / 144,774
Joined user–airing rows	3,014,129 (4,895 users, 4,363 programmes); 615,979 with screen time ≥ 0.8
Test users	1,260; 1,258 with labelled history; 1,150 with a full top-5 in weeks 21–30
Hold-out users (full top-5)	4,030; candidate pool 6,376 programmes; hybrid frame 763,617 rows
Test window	6,642 candidate programmes; 6,308 of 145,041 airings have an id never seen in weeks 11–30

Table 1. Data accounting.

Method (best setting, attempt)	Test users	All users	Time
Popularity baseline (#1)	0.305	0.279	<1 s
CBF TF-IDF, channel weight 2 (#89)	0.204	0.190	2 s
Item-kNN, shrink 50 (#92)	0.218	0.207	1 s
ALS f=16 (#66); f=64 (#6)	0.235; 0.212	0.227; 0.208	2 s
Association rules, confidence, minsup 50 (#82)	0.320	0.300	3 s
Personal frequency, τ = 0.5 (#61); τ = 0.8 (#2)	0.392; 0.383	0.356; 0.347	<1 s
PPR α = 0.5 (#77); α = 0.15 (#8)	0.394; 0.349	0.361; 0.323	30 s
Hybrid LightGBM lambdarank, 1 split (#11)	0.429	0.398	59 s (CV)
Hybrid LightGBM binary, 3 seeds (#22)	0.439	0.407	115 s (CV)
Hybrid LightGBM binary, 3 splits + lags, 5 seeds (#107)	0.446	0.412	529 s (CV)
Final ensemble (#123)	**0.447**	**0.413**	742 s (train+score)

Table 2. Hold-out MAP@5 (history weeks 11–20 → truth weeks 21–30). All 124 attempts are listed in results/attempts_table.csv.

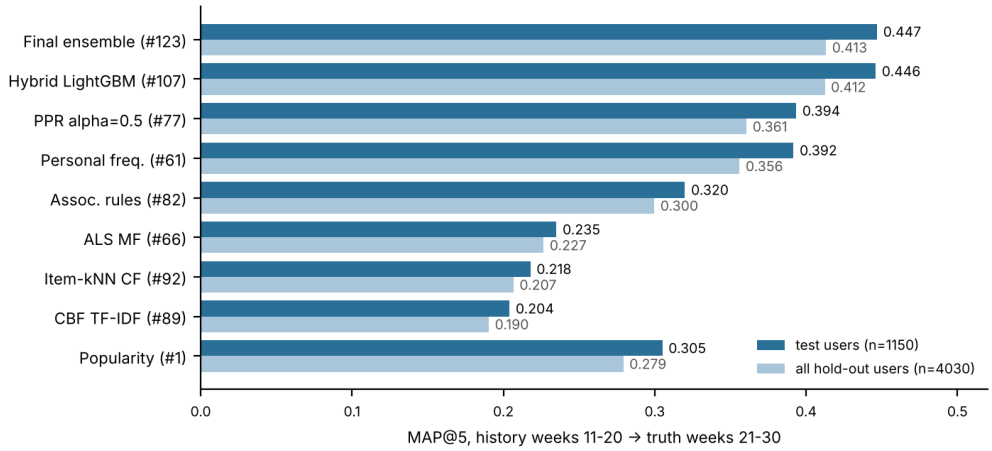


Fig. 1. Best setting of each method on the same hold-out, the same candidate pool and the same popularity fallback.

Comparison vs baseline. Popularity is a strong reference (0.305), because a few national news and morning shows dominate everyone's top-5. ALS, item-kNN and CBF fall below it: they rank programmes *similar* to the history, while the answer mostly contains the *same* programmes again. Personal frequency and PPR (whose restart keeps most mass on the user's own items when $\alpha \geq 0.5$) beat popularity by about 0.09. The hybrid adds a further +0.053, because it learns when a habit continues: recent weekly counts, catch rate and the number of target-window airings. Hyper-parameter changes after attempt 35 moved the score by at most ±0.005, the size of the noise.

Kaggle. The first hybrid submission (`bin3`: binary objective, 3 seeds, no extra splits, i.e. attempt 22 on the hold-out) returned private 0.44756 / public 0.45892. The final ensemble returned **private 0.44801 / public 0.46135** (Fig. 2), the best of the two on both splits (the course grades the private score), with the description "Емельянов Сергей Борисович (pochtasergeia) — …" (full name in Russian and Kaggle name, as the course requires), and it is the designated final submission (Kaggle disables selecting a submission after the deadline, so it is designated by this screenshot, this report and submission/sub_final.csv in the repository).

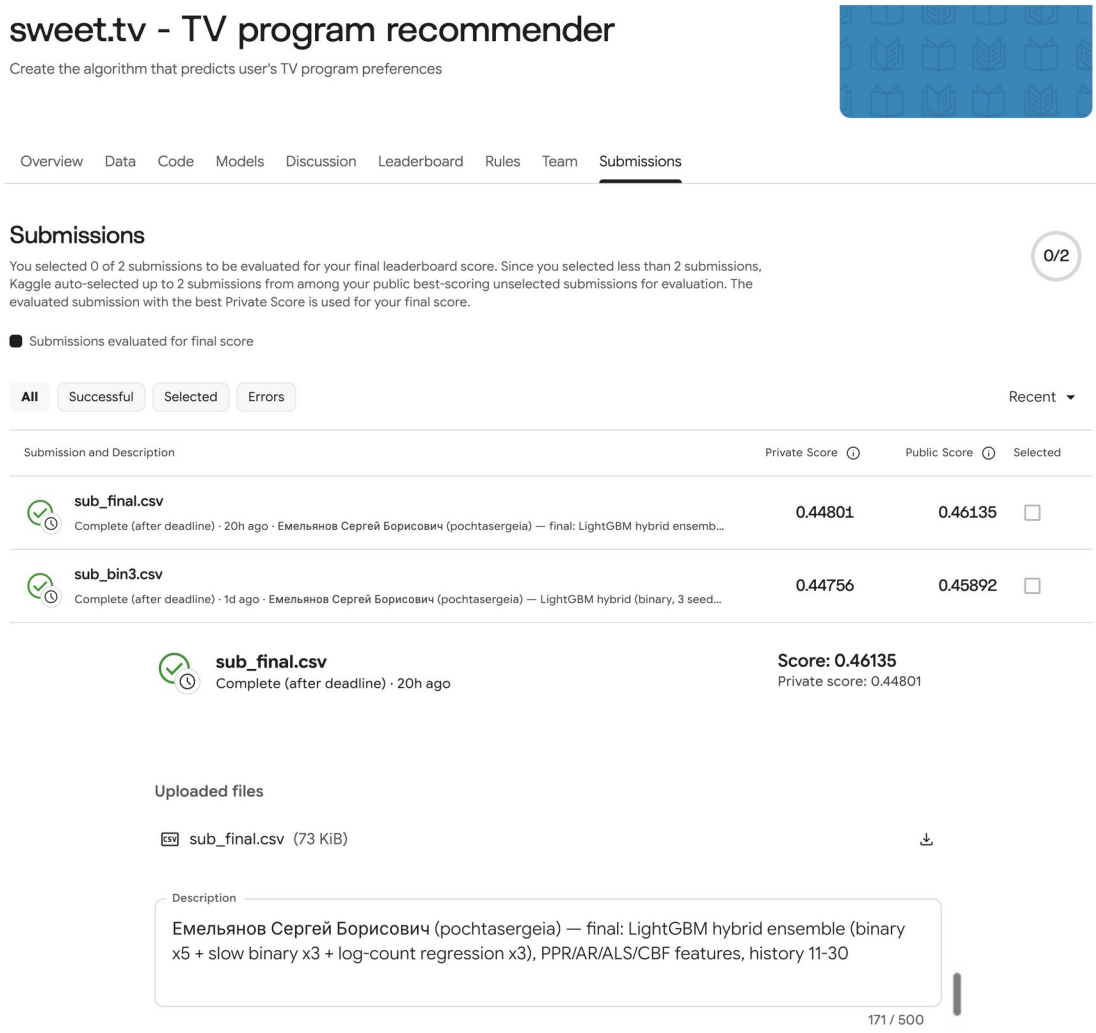


Fig. 2. Kaggle, account pochtasergeia. Top: submissions page with both late submissions; the final sub_final.csv returned private 0.44801 and public 0.46135. Bottom: its details with the description (full name and Kaggle name).

Verification. (a) *Join on one data point*: on 2020-03-16 the user above viewed channel 1+1 from 06:26:04 to 06:27:24, while ТСН. Неделя aired 05:00–06:30 (5,400 s). Overlap = 80 s, screen time 80 / 5,400 = 0.0148, so this airing is not counted; the 12:00 airing of the same show (1,200 s) lies inside the view 11:57:48–12:28:02, overlap 1,200 s, screen time 1.0, counted. Both rows are in the join output (`ts` 622800 and 648000). (b) *Metric on one data point*: popularity for the same user hits ranks 1, 2, 4 and 5, so AP@5 = (1/1 + 2/2 + 3/4 + 4/5) / 5 = 3.55 / 5 = 0.710, equal to `apk` in Table 3; on the slide example `apk` returns 0.9167 and 0.5333. (c) *Answer rule, independent re-implementation*: a separate script that does not import `src/` joins each sampled user's views to every airing of the channel by brute force; on 300 random users it gives identical top-5 lists for 247 of 247 users with a full top-5. Cutting the window by schedule file instead of by start time changes one of the 247 (user 5215032701573495051: four airings of Свинка Пеппа on 2020-05-18 sit in the weeks 11–20 file, 283 labelled week-21 airings in total). (d) *Submission file*: 1,260 rows in the template's user order, five distinct non-zero ids that air in weeks 31–42 (`checks/check_submission.py`). (e) *Leakage*: features read only history-window views and the target-window *schedule*; the CV folds are grouped by user.

Reproduction on a second machine (Table 4). The pipeline was re-run from a fresh clone twice on Linux x86-64 (by Claude Code) and once on my Apple M1 in a separate OpenCode session that I directed, with the same package versions. On Linux every number and the submission file are identical. On the M1, numpy uses Apple Accelerate instead of OpenBLAS; the last bits of the sparse AR and CBF products change, which reorders near-ties, gives a hybrid candidate frame of 763,658 rows instead of 763,617 and shifts LightGBM's splits.

Check	Linux x86-64, OpenBLAS	Apple M1, Accelerate (my session)
Answer rule, independent script (300 users)	247 / 247	247 / 247
10 single methods vs the log	10 / 10 to 5 decimals	7 / 10; AR and CBF within 0.00053
Hybrid #107 / #121 / #122 (test users)	0.44603 / 0.44511 / 0.44511	0.44565 / 0.44678 / 0.44408
Final ensemble #123 (test / all users)	0.44692 / 0.41315	0.44548 / 0.41138
Candidate recall of the true top-5	0.8515	0.8513
Regenerated sub_final.csv	byte-identical (sha256 76f439ce… 7fe8)	all format checks pass; same five ids for 1,037 / 1,260 users, same order for 722

Table 4. Reproduction from a fresh clone (logs in results/oc/ of the OpenCode session).

Recommendation examples (same two users for every method)

I chose a typical heavy viewer (90th percentile of ≥ 0.8 airings in weeks 11–20, 179 airings) and a light viewer (5th percentile, 12 airings), both test users, to show where each method works. ✔ = in the user's actual weeks 21–30 top-5. Titles and categories are looked up from the schedule.

Method	AP@5	Top-5 (id title [category]) — heavy user 8377619604347126107
Popularity	0.710	2400480 ТСН. Неделя [Инфо] ✔; 240081 Факты. День [Инфо] ✔; 12001682 Губка Боб [Детям]; 20088 Завтрак с 1+1 [Развлечения] ✔; 2400508 Сегодня. Итоги [Инфо] ✔
Personal freq.	0.760	ТСН. Неделя ✔; Факты. День ✔; 12002955 Свинка Пеппа [Детям] ✔; 200338 Готовим вместе [Познавательное]; Завтрак с 1+1 ✔
ALS (MF)	0.550	Свинка Пеппа ✔; ТСН. Неделя ✔; 200344 Чистоnews [Развлечения]; Завтрак с 1+1 ✔; 12002896 Вспыш и чудо-машинки [Детям]
PPR	0.800	ТСН. Неделя ✔; Факты. День ✔; Сегодня. Итоги ✔; Завтрак с 1+1 ✔; 2400467 Викна-Новости [Инфо]
Assoc. rules	0.550	ТСН. Неделя ✔; Свинка Пеппа ✔; Вспыш и чудо-машинки [Детям]; Факты. День ✔; Викна-Новости [Инфо]
CBF	0.167	5900319 Абзац! [Инфо]; Викна-Новости [Инфо]; ТСН. Неделя ✔; Факты. День ✔; 2500261 Подробности недели [Инфо]
Hybrid (final)	1.000	ТСН. Неделя ✔; Факты. День ✔; Завтрак с 1+1 ✔; Свинка Пеппа ✔; Сегодня. Итоги ✔
Method	**AP@5**	**Top-5 — light user 795534560850101267**
Popularity	0.400	ТСН. Неделя ✔; Факты. День ✔; Губка Боб [Детям]; Завтрак с 1+1 [Развлечения]; Сегодня. Итоги [Инфо]
Personal freq.	0.333	Факты. День ✔; 200337 Утро с Интером [Развлечения]; ТСН. Неделя ✔; 500281 Говорит Украина [Развлечения]; 5900348 Украинские сенсации [Инфо]
ALS (MF)	0.400	Факты. День ✔; ТСН. Неделя ✔; 6200371 ДжеДАИ [Познавательное]; Утро с Интером; Чистоnews
PPR	0.400	ТСН. Неделя ✔; Факты. День ✔; Губка Боб; Утро с Интером; Викна-Новости
Assoc. rules	0.333	ТСН. Неделя ✔; Чистоnews; Факты. День ✔; Сегодня. Итоги; Викна-Новости
CBF	0.400	ТСН. Неделя ✔; Факты. День ✔; Подробности недели [Инфо]; Абзац! [Инфо]; Украинские сенсации [Инфо]
Hybrid (final)	0.400	Факты. День ✔; ТСН. Неделя ✔; Викна-Новости; Губка Боб; Сегодня. Итоги

Table 3. Top-5 of every method on the hold-out (results/examples.md). Actual top-5, heavy: ТСН. Неделя, Сегодня. Итоги, Факты. День, Завтрак с 1+1, Свинка Пеппа; light: Факты. День, 200352 Вечерний квартал, ТСН. Неделя, two films (90018819 Сломанная стрела, 90080730 Борг/Макинрой).

5. Discussion — which strategy for SWEET.TV, and why

Recommendation. Use the hybrid as the main "for you" row: candidates from the user's own history, PPR and association rules, re-ranked by LightGBM. It scores 0.447 against 0.394 for the best single method and 0.305 for popularity on the same users and protocol. Single methods cost seconds; the whole final model trains and scores 1,260 users in 742 s on 4 vCPUs. Retraining weekly is therefore cheap, and serving is a feature lookup plus a tree ensemble over about 190 candidates per user.

How the methods differ (Table 3). For the heavy user, personal frequency keeps *Готовим вместе*, which the user stopped watching. PPR's diffusion swaps it for *Сегодня. Итоги*, a show co-watched by similar viewers, which was in the truth. The hybrid also uses the target-window schedule and recent weeks, and it recovers all five. Association rules and ALS see the kids' cluster (Свинка Пеппа → Вспыш) and so capture household co-viewing. CBF only matches attributes (news → news), which helps where co-view signal is thin. For the light user every method shares the same two hits, the national news shows that popularity already provides. Only three of the five true programmes are among the user's 192 candidates: the two films are not, so 0.600 is the ceiling, and the hybrid reaches 0.400. The third reachable programme, Вечерний квартал, has a single 105 s touch of a 6,600 s airing in the history, too weak to rank it. The lists are mostly *familiar* programmes, which fits a "continue watching / your channels" row. A

discovery row (PPR beyond the user's own items, CBF for new-season series) should be a separate row: an unfamiliar item here is not shown to be a useful discovery.

Failure case. The first hybrid run reported candidate recall 0.0031 and MAP@5 0.305, the same as popularity, even though 85 % of true items were in the frame. **Root cause:** the 64-bit unsigned `user_id` hashes became `float64` after a `concat` with a frame of signed integers. Above 2^{53} the cast loses digits, so (user, item) keys no longer matched the truth dictionary and almost every label became 0. **Fix + verification:** `load()` maps ids to dense `int64` codes and keeps the raw id only for writing the CSV. Recall became 0.8515 and MAP@5 0.433 on the next run, and the written file's ids are diffed against the template (identical).

What worked. Reproducing the official answer rule before modelling; letting repeat viewing drive the model; user-grouped CV, which made 124 attempts comparable; re-running everything on a second machine, which separated the stable claims from the fragile ones.

What surprised me / didn't work. (1) PPR improved from 0.349 to 0.394 when the restart probability rose from 0.15 to 0.5, i.e. when it behaved *more* like personal frequency. (2) Generators find but cannot order: 78.6 % of a test user's true top-5 is already in their own weeks 11–20 history, and ALS holds the most of it in its top-30 (recall 0.716, PPR 0.707, popularity 0.655) but the least in its top-5 (0.327 against 0.456 for PPR). So the generators are used only for candidates, and the re-ranker decides the order. (3) The ensemble's gain over its best component (+0.0009 over #107) is smaller than the spread between machines (up to 0.0018): on the M1, #121 alone scores above #123. Habit features, DART, lambdarank and extra hyper-parameter search did not beat the plain binary objective beyond noise.

Limitations supported by the results. Only 10.8 % of schedule rows carry an id, so most viewing time is invisible to every method. 85.2 % candidate recall caps the hybrid. With a perfect ordering of the candidates by true future count, MAP@5 is 0.851 when ties are broken as in my truth and 0.776 when they are broken randomly; part of the score therefore depends on the organisers' unpublished tie rule. 4.3 % of test-window airings carry programme ids never seen before (new season), which the hybrid can reach only through popularity or CBF. The defensible claim is the ordering hybrid > single methods > popularity, which holds on both machines and on Kaggle (one concordant pair: +0.00045 private), not the third decimal. The log covers this platform's views only, so absence from it does not mean a user never saw a programme, and nothing here shows that users enjoyed what they watched. No online test was run, and no revenue claim is made.

Next improvement. An airing-level model that predicts P(screen time ≥ 0.8) for every target-window airing and sums the predictions into an expected count per programme. This would handle long films and schedule changes directly.

6. AI Usage Disclosure

- **Tools.** (1) Claude Code (Anthropic), cloud session, model Claude Opus 5.5, working on my Mac folder ~/kaggle and in my Chrome session (Kaggle rules acceptance, data download, the two late submissions). (2) OpenCode v2.1.2 [12] with GLM-5.3-Flash (Z.AI Coding Plan), on my Apple M1 (macOS 26.3.1), for a separate reproduction session.
- **How AI was used.** Claude Code: reading the task and the Data page; code generation (all of `src/`); running the 124 attempts; debugging (the float64 id cast); the figure, the README and the drafts of this report (English and Russian); the plan of the OpenCode session (prompt sequence, expected numbers, reply templates) and its four check scripts (`audit_data.py`, `independent_truth.py`, `hand_inputs.py`, `check_submission.py`); during that session, reviewing the agent's answers and drafting most of my replies to it. OpenCode: re-running the pipeline from a fresh clone on my machine and checking each number (27 prompts, 97 tool calls, 2.9 h).
- **What I personally verified.** (a) I re-ran the experiment from scratch on my own machine and compared every number with the logged one (Table 4); the regenerated file is not byte-identical on Apple Silicon, and I recorded that instead of replacing the submitted file. (b) I read the private and public scores of both submissions on the Kaggle page myself (Fig. 2) and checked them against the README. (c) The hold-out order of the two submitted configurations (#123 0.447 > #22 0.439) matches their Kaggle order on both splits.
- **Trusted without verification / limits of my checks.** The data-point checks in §4 (a, b) were worked out with Claude; the same values appear in my OpenCode turns, which I asked Claude to write, so they are an AI cross-check confirmed by the code, not an independent hand calculation of mine. Two errors of the OpenCode agent (a table cell with 0.40989 instead of 0.44511 for #122, and an airing length of 2,700 s instead of 6,600 s) were found by Claude when I asked it to review the agent's answers; the correction of my mapping of the first submission to attempt #22 came from the OpenCode agent. The byte-identical runs on Linux were done by Claude Code, not on my machine. I also trusted the internals of LightGBM and `implicit` ALS, and the organisers' description of the answer rule (their tie-breaking is unknown).
- **Session logs.** Both sessions are submitted unmodified: `emelyanov_sergey_exam_session.jsonl` (Claude Code: development, the 124 attempts, both submissions, this report) and `emelyanov_sergey_exam_opencode_session.json` (OpenCode session ses_e95aa2d45ffelUbzQd00ghZrWQ of 2026-10-23, the reproduction in Table 4; its logs in `results/oc/` are not committed, this report carries their numbers).

References

[1] ogurbych, "sweet.tv – TV program recommender," Kaggle Community Prediction Competition, 2020. [Online]. Available: https://www.kaggle.com/competitions/sweettv-tv-program-recommender. [Accessed: 2026-10-22].

[2] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative filtering for implicit feedback datasets," in Proc. 8th IEEE Int. Conf. Data Mining (ICDM), 2008, pp. 263–272, doi: 10.1109/ICDM.2008.22.

[3] T. H. Haveliwala, "Topic-sensitive PageRank," in Proc. 11th Int. World Wide Web Conf. (WWW), 2002, pp. 517–526, doi: 10.1145/511446.511513.

[4] R. Agrawal and R. Srikant, "Fast algorithms for mining association rules," in Proc. 20th Int. Conf. Very Large Data Bases (VLDB), 1994, pp. 487–499.

[5] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Item-based collaborative filtering recommendation algorithms," in Proc. 10th Int. World Wide Web Conf. (WWW), 2001, pp. 285–295, doi: 10.1145/371920.372071.

[6] P. Lops, M. de Gemmis, and G. Semeraro, "Content-based recommender systems: State of the art and trends," in Recommender Systems Handbook, F. Ricci et al., Eds. Boston, MA: Springer, 2011, pp. 73–105, doi: 10.1007/978-0-387-85820-3_3.
[7] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 3146–3154.
[8] C. J. C. Burges, "From RankNet to LambdaRank to LambdaMART: An overview," Microsoft Research, Tech. Rep. MSR-TR-2010-82, 2010.
[9] B. Hamner, "Metrics: Machine learning evaluation metrics (average_precision.py)," GitHub repository. [Online]. Available: https://github.com/benhamner/Metrics. [Accessed: 2026-10-22].
[10] B. Frederickson, "implicit: Fast Python collaborative filtering for implicit datasets," GitHub repository. [Online]. Available: https://github.com/benfred/implicit. [Accessed: 2026-10-22].
[11] Microsoft, "LightGBM documentation." [Online]. Available: https://lightgbm.readthedocs.io. [Accessed: 2026-10-22].
[12] OpenCode, "OpenCode documentation." [Online]. Available: https://opencode.ai/docs/. [Accessed: 2026-10-24].